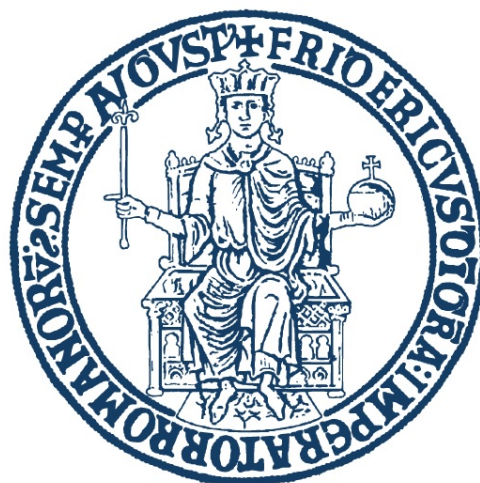




UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II

UNIVERSITÀ DEGLI STUDI DI NAPOLI FEDERICO II
CORSO DI LAUREA TRIENNALE IN INFORMATICA

Labyrinth Game

Documentazione Sistema Client-Server

Laboratorio di Sistemi Operativi

Mario Majorano	N86005035
Luca Sanselmo	N86005147

Anno Accademico 2025/2026

Indice

1	Introduzione	3
1.1	Descrizione del Progetto	3
1.2	Obiettivi del Sistema	3
2	Guida d'uso	4
2.1	Requisiti e Compilazione Classica	4
2.1.1	Struttura delle directory	4
2.1.2	Direttive di compilazione	4
2.1.3	Avvio del Server	4
2.1.4	Avvio del Client	5
2.2	Requisiti e Compilazione con Docker	5
2.2.1	Cenni Teorici	5
2.2.2	Motivazione dell'utilizzo di Docker	5
2.2.3	Struttura dei file Docker	6
2.2.4	Costruzione delle immagini e dei container	6
2.2.5	Avvio del Server	6
2.2.6	Avvio del Client	6
2.2.7	Fase di Teardown	7
2.3	Comandi Interattivi	7
2.3.1	Fase di Lobby	7
2.3.2	Fase di Gioco	7
3	Protocollo Applicativo	8
3.1	Glossario dei Messaggi	8
3.1.1	Messaggi Client → Server	8
3.1.2	Messaggi Server → Client	8
3.2	Fase di Lobby e Avvio Sincronizzato della Partita	9
3.3	Le Fasi del Thread di un Client	9
3.4	Il Protocollo della Lobby	11
4	Meccanismo di Autenticazione	13
4.1	Cenni Teorici	13
4.2	Implementazione	14
4.3	Sicurezza: Funzione di Hash	14
5	Architettura del Sistema	16
5.1	Modello di Concorrenza	16
5.2	Condivisione e Sincronizzazione delle Strutture	17
5.2.1	Struttura Dati dei Giocatori	17
5.2.2	Struttura dei Mutex	19
5.3	Meccanismo di Notifica tra Thread	20
5.3.1	Il problema: eventi esterni durante l'attesa in select()	20
5.3.2	La soluzione: un self-pipe condiviso	20
5.3.3	Il problema: sincronizzazione di fine partita	22
5.3.4	La soluzione: le condition variable	22
5.3.5	Schema riassuntivo: meccanismi di sincronizzazione tra thread	26

6	Dettagli Implementativi	27
6.1	Definizioni Formali	27
6.1.1	Livello Logico: Struttura del Labirinto	28
6.1.2	Livello Fisico: Rappresentazione in Memoria	29
6.2	Generazione del Labirinto	30
6.2.1	Algoritmo di Prim	30
6.2.2	Fondamenti Teorici	30
6.2.3	Descrizione dell'Algoritmo	30
6.2.4	Pseudocodice	31
6.2.5	La Frontiera: Definizione Formale e Ruolo	32
6.2.6		33
6.3	Mappa Locale e Mappa Globale	36
6.3.1	Differenza Concettuale	36
6.3.2	Costruzione della Mappa Locale	36
6.3.3	Rivelazione delle Celle	38
6.4	Invio Periodico della Mappa Globale: Il Meccanismo del Timeout	39
6.4.1	Il Problema: Tre Sorgenti di Input Asincrone	39
6.5	Meccanica di Punteggio e Interazione con la Griglia	42
6.5.1	Movimento e Validazione	42
6.5.2	Raccolta degli Oggetti	43
6.6	Raggiungimento di un'uscita	43
6.7	Logica di Fine Partita: Implementazione della Verifica	44
6.7.1	Funzionamento informale della verifica	44
6.7.2	Implementazione	45
6.7.3	Cleanup, Disconnessione e Rigenerazione del Labirinto	47
6.8	Logging Thread-Safe	49
7	Rendering del Client	50
7.1	Colori ANSI	50
7.2	Terminale in Modalità Raw e Schermo Alternativo	50
7.3	Gestione Asincrona dei Messaggi	51
8	Parametri di Configurazione	51
9	Repository GitHub	52

1 Introduzione

1.1 Descrizione del Progetto

Il presente progetto ha l'obiettivo di realizzare un sistema **client-server** che consente a più utenti di partecipare simultaneamente ad un gioco di esplorazione di un labirinto generato casualmente. Il sistema è sviluppato interamente in linguaggio **C** su piattaforma **UNIX/Linux**, con comunicazione tramite **socket TCP**.

1.2 Obiettivi del Sistema

- Gestione concorrente di più client tramite **thread POSIX** (`pthread_create()`)
- Sincronizzazione dello stato condiviso tramite `pthread_mutex_t`
- Registrazione e autenticazione degli utenti (file `users.txt`)
- Generazione casuale del labirinto con algoritmo di **Prim**
- Invio periodico della mappa globale tramite `select()` con timeout
- Determinazione del vincitore secondo le regole della specifica
- Logging thread-safe completo su file con timestamp ISO-8601
- Rendering grafico su terminale con colori ANSI e simboli Unicode

2 Guida d'uso

2.1 Requisiti e Compilazione Classica

Il progetto richiede un sistema UNIX/Linux con compilatore `gcc`, supporto POSIX threads (`-pthread`) e libreria C standard. Non sono richieste librerie esterne.

2.1.1 Struttura delle directory

```
1 Labirinto/  
2 |-- Makefile  
3 |-- common/  
4 |   |-- protocol.h           #costanti e protocollo condiviso  
5 |-- server/  
6 |   |-- server.c             #server, pthread, stato globale  
7 |   |-- maze.c / maze.h      #generazione e gestione labirinto  
8 |   |-- player.c / player.h  #stato per-giocatore, mappe  
9 |   |-- game.c / game.h      #logica fine partita e vincitore  
10 |   |-- logger.c / logger.h #logging thread-safe su file  
11 |-- client/  
12 |   |-- client.c             #connessione, auth, loop di gioco
```

Listing 1: Struttura del progetto

2.1.2 Direttive di compilazione

Nella cartella di download, eseguire:

```
1 make          # compila server e client  
2 make clean    # rimuove file oggetto e binari
```

Listing 2: Compilazione con make

Nota

Il server deve essere compilato con il flag `-pthread` per abilitare il supporto ai thread POSIX. Il Makefile si occupa di includere automaticamente questo flag.

2.1.3 Avvio del Server

```
1 ./server/server.out <porta>  
2 # Esempio:  
3 ./server/server.out 5000
```

Listing 3: Avvio del server

Nota

Il server non produce output su `stdout`. Tutte le attività vengono registrate nel file `server.log` nella directory corrente. In caso di errore fatale, un messaggio viene stampato su `stderr`.

2.1.4 Avvio del Client

```
1 ./client/client.out <host> <porta>
2 ./client/client.out localhost 5000
3 ./client/client.out 192.168.1.10 5000
```

Listing 4: Avvio del client

2.2 Requisiti e Compilazione con Docker

Il progetto richiede il software **Docker** per l'orchestrazione dei container.

2.2.1 Cenni Teorici

Definizione

Docker è una piattaforma di containerizzazione che consente di **creare, distribuire ed eseguire applicazioni all'interno di container**, ovvero ambienti isolati che condividono il kernel del sistema operativo host ma mantengono **separati il filesystem, i processi e le dipendenze dell'applicazione**.

2.2.2 Motivazione dell'utilizzo di Docker

L'utilizzo di **Docker** è stato introdotto con l'obiettivo di garantire un ambiente di esecuzione **riproducibile, isolato e facilmente configurabile** per il progetto.

La containerizzazione consente di racchiudere l'applicazione, le relative dipendenze e gli strumenti necessari all'esecuzione all'interno di un ambiente controllato, riducendo la dipendenza dalla configurazione specifica della macchina host.

In particolare, Docker permette di:

- **standardizzare l'ambiente di esecuzione**, assicurando che il progetto venga eseguito con le medesime configurazioni indipendentemente dal sistema utilizzato;
- **isolare l'applicazione** dal sistema operativo host e dagli altri servizi presenti sulla macchina;
- **semplificare la configurazione e l'avvio del progetto**, evitando l'installazione manuale delle dipendenze richieste;
- **rendere riproducibile l'ambiente di sviluppo e testing**, facilitando la collaborazione tra i membri del gruppo;
- **agevolare la distribuzione del progetto**, poiché l'ambiente necessario all'esecuzione può essere ricreato a partire dalla configurazione Docker fornita.

Docker non costituisce quindi una componente necessaria al funzionamento logico dell'applicazione, ma rappresenta uno strumento di **supporto all'esecuzione e alla distribuzione**, finalizzato principalmente a garantire portabilità, isolamento e riproducibilità dell'ambiente.

Di seguito la struttura e le direttive definite per il corretto avvio del programma:

2.2.3 Struttura dei file Docker

```
1 Labirinto/  
2 |-- docker-compose.yml      #file di orchestrazione completa  
3 |-- .dockerignore           #elenco di file ignorati da Docker  
4 |-- common/  
5 |   |-- protocol.h          #costanti e protocollo condiviso  
6 |-- server/  
7 |   |-- Dockerfile           #file di creazione immagine server  
8 |   |-- *.c                  #file .c e .h visti in precedenza  
9 |-- client/  
10 |   |-- Dockerfile           #file di creazione immagine client  
11 |   |-- client.c             #file .c visto in precedenza
```

Listing 5: Struttura del progetto

2.2.4 Costruzione delle immagini e dei container

Per creare le immagini necessarie all'avvio del programma, sarà sufficiente eseguire nella cartella di download:

```
1 docker compose build
```

Listing 6: Creazione immagini

Docker si occuperà di generare le immagini da cui sarà possibile lanciare i container.

2.2.5 Avvio del Server

```
1 docker compose up -d server
```

Listing 7: Avvio del server

Nota

Il flag `-d` consente di avviare il server in modalità *detached*, ovvero in background.

2.2.6 Avvio del Client

```
1 docker compose run --rm client
```

Listing 8: Avvio del client

Nota

Il flag `--rm` rimuove automaticamente il container del client una volta terminata l'esecuzione

2.2.7 Fase di Teardown

Al termine dell'utilizzo del programma, si eseguirà questo comando per terminare il container server in background:

```
1 docker compose down
```

Listing 9: Teardown server

Nota

Aggiungendo il flag `-v` sarà possibile rimuovere anche il volume dati che conserva le informazioni degli utenti e i log di gioco.

2.3 Comandi Interattivi

2.3.1 Fase di Lobby

Tasto	Comando inviato	Effetto
r	READY	Dichiara il giocatore pronto a iniziare
q	QUIT	Disconnessione dalla lobby

2.3.2 Fase di Gioco

Tasto	Comando inviato	Effetto
w	MOVE N	Muove verso Nord
a	MOVE W	Muove verso Ovest
s	MOVE S	Muove verso Sud
d	MOVE E	Muove verso Est
l	LIST	Mostra giocatori connessi
q	QUIT	Disconnessione dalla partita

Nota

Durante la fase di lobby i tasti di movimento non sono attivi: il client accetta solo **r** (pronto) e **q** (esci). Analogamente, a partita iniziata il tasto **r** non ha più effetto poiché la fase di lobby è terminata.

3 Protocollo Applicativo

Il protocollo è di tipo **testuale** e *line-oriented* a livello applicativo, costruito sulla struttura TCP/IP.

Definizione

Un **messaggio** del protocollo è una riga di testo terminata dal carattere '**\n**':

COMANDO [**arg₁**] [**arg₂**] ... '**\n**'

dove **COMANDO** è una stringa ASCII maiuscola e gli argomenti sono opzionali.

3.1 Glossario dei Messaggi

3.1.1 Messaggi Client → Server

Messaggio	Significato
REGISTER <nick> <pass>	Registra un nuovo utente. Risposta: OK o ERR nick already exists .
LOGIN <nick> <pass>	Autentica un utente esistente. Risposta: OK , ERR invalid credentials o ERR already logged in .
MOVE <dir>	Movimento in direzione N, S, E o W.
LIST	Lista giocatori connessi.
QUIT	Disconnessione volontaria.
AGAIN	Client intenzionato a rigiocare: inviato automaticamente dopo GAME_END ; riporta il giocatore in phase_join per una nuova partita senza chiudere la connessione TCP.

3.1.2 Messaggi Server → Client

Messaggio	Significato
OK	Conferma generica.
ERR <messaggio>	Errore con descrizione testuale.
LOCAL <r> <c> <data>	Mappa locale 3 × 3 centrata sul giocatore.
GLOBAL <r> <c> <data>	Mappa globale mascherata 20 × 20.
COLLECT <score>	Notifica raccolta oggetto.
EXIT_OK <score>	Notifica che il giocatore ha raggiunto l'uscita.
LIST <count> <nick1> <nick2> ...	Risposta a LIST : numero di giocatori connessi seguito dai relativi nickname.
GAME_END WIN <nick> <score>	Fine partita con vincitore unico.
GAME_END DRAW <score>	Fine partita con pareggio.

3.2 Fase di Lobby e Avvio Sincronizzato della Partita

Un client autenticato non entra immediatamente in partita, ma resta in attesa finché non dichiara di essere pronto e finché non lo sono anche gli altri giocatori connessi. In tale fase subentra la **Lobby**.

Definizione

La **lobby** è la fase del protocollo durante la quale i giocatori autenticati possono connettersi alla partita e attendere l'avvio del gioco. In tale fase ogni giocatore può dichiararsi pronto mediante il comando **READY**, mentre il server mantiene il numero dei giocatori connessi e di quelli che sono già **READY**.

3.3 Le Fasi del Thread di un Client

Ogni thread dedicato a un client attraversa, in sequenza, quattro funzioni distinte, racchiuse in un ciclo esterno che permette la **rivincita** (**AGAIN**) senza chiudere la connessione TCP:

```
1 void* handle_client(void* arg) {
2     ClientArgs args = *(ClientArgs*)arg;
3     free(arg);
4
5     int fd = args.fd;
6     struct sockaddr_in addr = args.addr;
7
8     char nick[MAX_NICK_LEN] = {0};
9     int player_idx = -1;
10    int player_ready = 0;
11
12    if (phase_auth(fd, addr, nick, sizeof(nick))) {
13        player_idx = phase_join(fd, nick);
14        if (player_idx >= 0) {
15            if (phase_lobby(fd, &player_ready)) {
16                if (phase_wait_start(fd)) {
17                    phase_play(fd, player_idx, nick);
18                }
19            }
20        }
21    }
```

Listing 10: Sequenza delle fasi in `handle_client`, incluso il supporto ad **AGAIN** (`server.c`)

```

1      /* ciclo di rivincita: alla ricezione di AGAIN si
        rientra in lobby */
2      while (1) {
3          char buf[MAX_MSG_LEN];
4          client_cleanup(player_idx, &player_ready);
5          if (recv_line(fd, buf) < 0) { close(fd); return
            NULL; }
6          if (strcmp(buf, "AGAIN") == 0) {
7              player_idx = phase_join(fd, nick);
8              if (player_idx >= 0) {
9                  if (phase_lobby(fd, &player_ready)) {
10                     if (phase_wait_start(fd)) {
11                         phase_play(fd, player_idx, nick);
12                     }
13                 }
14             }
15         } else break;
16     }
17
18     log_write("DISCONNECT nick=%s", nick);
19     close(fd);
20     return NULL;
21 }

```

Fase	Responsabilità
<code>phase_auth</code>	Accetta solo <code>REGISTER</code> / <code>LOGIN</code> finché il client non è autenticato.
<code>phase_join</code>	Cerca una cella libera casuale e uno slot in <code>PlayerTable</code> ; rifiuta con <code>ERR server full</code> se non ce ne sono, o con <code>ERR game already in progress</code> se la partita è già iniziata.
<code>phase_lobby</code>	Attende il comando <code>READY</code> (o <code>QUIT</code>), incrementando <code>pt.ready_count</code> .
<code>phase_wait_start</code>	Sondaggio (<code>select()</code> con timeout di 1s) in attesa che <code>maze.game_started</code> diventi vero, restando comunque ricettivo a <code>QUIT</code> .
<code>phase_play</code>	Ciclo di gioco vero e proprio.
<code>client_cleanup</code>	Rilascia lo slot del giocatore in <code>PlayerTable</code> e sveglia gli altri thread; <i>non</i> chiude il socket, per permettere l'eventuale rivincita.

Nota

A differenza di una singola partita "usa e getta", il thread del client **non termina** subito dopo `phase_play()`: rientra in un ciclo che chiama `client_cleanup()` (rilasciando lo slot occupato in partita) e si mette in attesa di un nuovo comando sullo stesso socket. Se il client invia **AGAIN**, il thread ripete l'intera sequenza `phase_join` → `phase_lobby` → `phase_wait_start` → `phase_play` riutilizzando le stesse credenziali (`nick`) già autenticate in precedenza, senza dover rifare il login. Qualsiasi altro messaggio (compresa una disconnessione) fa uscire dal ciclo e termina definitivamente il thread, chiudendo il socket solo a questo punto.

3.4 Il Protocollo della Lobby

Messaggio	Significato
WAITING <ready>/<count>	(Server→Client) Inviato all'ingresso in lobby e ad ogni variazione: numero di giocatori pronti su numero di giocatori connessi. Trasmesso in broadcast a tutti i client ancora in lobby.
READY	(Client→Server) Il giocatore dichiara di essere pronto a iniziare.
START	(Server→Client) Inviato non appena la partita parte; seguito immediatamente da un messaggio LOCAL con la vista iniziale del giocatore.

```

1  if (strcmp(cmd, "READY") == 0) {
2      pthread_mutex_lock(&mutex);
3      pt.ready_count++;
4      *player_ready = 1;
5      broadcast_lobby_update();
6      if (pt.ready_count >= pt.count && pt.count >= 2) {
7          maze.game_started = 1;
8          maze.start_time = time(NULL);
9      }
10     pthread_mutex_unlock(&mutex);
11     return 1;
12 }

```

Listing 11: Gestione del comando READY e regola di avvio (server.c)

Regola di Avvio Partita

La partita inizia (`maze.game_started = 1`) $\iff$ **tutti** i giocatori attualmente connessi sono in stato di **READY** (`ready_count  $\geq$  count`) e sono presenti **almeno due** giocatori (`count  $\geq$  2`). Un singolo giocatore pronto non può quindi avviare la partita da solo.

Formalmente, sia

$$P = \{p_1, \dots, p_n\}$$

l'insieme dei giocatori attualmente connessi e sia

$$R = \{p \in P \mid p \text{ ha inviato il comando } \text{READY}\}$$

l'insieme dei giocatori che hanno dichiarato di essere pronti. La partita inizia se e solo se

$$|R| = |P| \quad \wedge \quad |P| \geq 2.$$

Equivalentemente, indicando con $N = |P|$ il numero di giocatori connessi e con $R = |R|$ il numero di giocatori pronti, la condizione di avvio può essere espressa come

$$R = N \quad \wedge \quad N \geq 2.$$

Nota

La transizione di `maze.game_started` è osservata da tutti i thread tramite `phase_wait_start()`, che effettua un *polling* ogni secondo (`select()` con timeout fisso di 1s sul solo socket del client, per restare comunque reattivo a un eventuale **QUIT**). Non è quindi un meccanismo a evento immediato come quello descritto in §5.3 per la disconnessione, ma un'attesa attiva a bassa frequenza.

4 Meccanismo di Autenticazione

4.1 Cenni Teorici

Ogni utente è modellato nella piattaforma di gioco tramite due attributi fondamentali:

- Nome
- Password

Tale coppia **identifica un utente univocamente** nel contesto di gioco.

Formalmente, sia $\mathcal{U}$ l'insieme degli utenti registrati e sia

$$\mathcal{N}$$

l'insieme dei possibili nomi e

$$\mathcal{P}$$

l'insieme delle possibili password. Un utente $u \in \mathcal{U}$ è rappresentato dalla coppia

$$u = (n, p) \in \mathcal{N} \times \mathcal{P},$$

dove n rappresenta il nome dell'utente e p la relativa password.

La condizione di unicità del nome impone che, per ogni $u_1, u_2 \in \mathcal{U}$,

$$u_1 = (n_1, p_1) \wedge u_2 = (n_2, p_2) \wedge n_1 = n_2 \implies u_1 = u_2.$$

Pertanto, a ogni nome registrato corrisponde un unico utente all'interno del sistema.

4.2 Implementazione

Tale modellazione di un utente nel sistema di gioco è rappresentata tramite un meccanismo di registrazione aderente con quest'ultima. In particolare:

Ogni utente che desidera giocare necessita di effettuare una registrazione, indicando il nome e la propria password.

Successivamente, all'interno di un file nominato `users.txt` sarà così presente ogni giocatore sottoforma del seguente formato, in accordo con i dati inseriti dall'utente:

```
1 <nome> <password>
```

Listing 12: Esempio di riga in `users.txt`

4.3 Sicurezza: Funzione di Hash

Durante la fase di registrazione, prima della scrittura delle credenziali sul file `users.txt`, le password subiscono l'effetto di una funzione di hash non crittografica in stile *djb2*:

La funzione di hash utilizzata (*djb2*) è pensata per la distribuzione uniforme in tabelle hash, **non** per la sicurezza crittografica: non utilizza salt, è velocissima da invertire per forza bruta (in particolare su password brevi) ed è vulnerabile a collisioni.

Nota

Rispetto alla password in chiaro rappresenta comunque un miglioramento (il contenuto di `users.txt` non è leggibile a colpo d'occhio), ma per un uso reale andrebbe sostituita con una funzione dedicata e "salata".

```
1 unsigned long hash(char* str) {  
2     unsigned long hash = 5381;  
3     int c;  
4     while ((c = *str++))  
5         hash = ((hash << 5) + hash) + c;    /* hash * 33 + c  
6         */  
7     return hash;  
}
```

Listing 13: Funzione di hashing in `server.c`

Ogni riga di `users.txt` ha quindi il formato `<nome> <hashed_password>`, ad esempio:

```
1 mario 6385271281444
```

Listing 14: Esempio di riga in `users.txt`

```

1  int auth_login(const char* nick, char* pass) {
2      pthread_mutex_lock(&auth_mutex);
3      FILE* f = fopen(USERS_FILE, "r");
4      if (!f) { pthread_mutex_unlock(&auth_mutex); return -1; }
5
6      unsigned long hashed_pass = hash(pass);
7      char line[MAX_NICK_LEN + MAX_PASS_DIGITS + 1];
8      while (fgets(line, sizeof(line), f)) {
9          char n[MAX_NICK_LEN]; unsigned long p = 0;
10         if (sscanf(line, "%s %lu", n, &p) == 2 &&
11             strcmp(n, nick) == 0 && p == hashed_pass) {
12             /* impedisce un secondo login con lo stesso nick
13              * gia' attivo */
14
15             for (int i = 0; i < MAX_PLAYERS; i++)
16                 if (strcmp(pt.slots[i].nick, nick) == 0 &&
17                     pt.slots[i].active) {
18                     fclose(f);
19                     pthread_mutex_unlock(&auth_mutex);
20                     return -2; /* gia' connesso */
21                 }
22             fclose(f);
23             pthread_mutex_unlock(&auth_mutex);
24             return 0;
25         }
26     }
27     fclose(f);
28     pthread_mutex_unlock(&auth_mutex);
29     return -1;
30 }

```

Listing 15: Verifica delle credenziali in fase di LOGIN (server.c)

Nota

Il controllo sul secondo caso di ritorno (-2, “già connesso”) impedisce che lo stesso nickname occupi due slot contemporaneamente in `PlayerTable`: il server risponde in questo caso con **ERR already logged in**.

5 Architettura del Sistema

5.1 Modello di Concorrenza

Il server adotta un modello **thread-per-client**: per ogni nuova connessione accettata, il thread principale invoca `pthread_create()`, generando un thread POSIX dedicato a quel client.

Di seguito il loop principale per l'inizializzazione del server:

```
1 while (1) {
2     int client_fd = accept(listen_fd, &client_addr,
3                             &client_len);
4
5     ClientArgs *cargs = malloc(sizeof(ClientArgs));
6     cargs->fd = client_fd;
7     cargs->addr = client_addr;
8
9     pthread_t tid;
10    pthread_create(&tid, NULL, handle_client, cargs);
11    /* il thread verrà "staccato" con pthread_detach() */
12 }
```

Listing 16: Loop principale del server in server.c

Su ogni thread verrà chiamato `pthread_detach(tid)` dopo l'avvio, rendendo non necessario un successivo `pthread_join()` da parte del thread principale. Le risorse del thread vengono rilasciate automaticamente al termine.

Nota

A differenza del modello multi-processo, non è necessario gestire il segnale `SIGCHLD` né raccogliere processi zombie con `waitpid()`. I thread terminati e detachati vengono liberati automaticamente dal sistema operativo.

5.2 Condivisione e Sincronizzazione delle Strutture

Poiché tutti i thread condividono lo stesso spazio di memoria, le strutture dati del gioco vivono come variabili globali del processo.

L'accesso concorrente è **protetto da mutex POSIX**.

5.2.1 Struttura Dati dei Giocatori

La Struttura Player

```
1 typedef struct {
2     int active;                // 1 = slot occupato
3     pthread_t tid;             // TID del thread proprietario
4     char nick[MAX_NICK_LEN];
5     int row, col;              // posizione corrente
6     int score;                 // oggetti raccolti
7     int exited;
8     int discovered[MAZE_ROWS][MAZE_COLS];
9 } Player;
10
11 typedef struct {
12     Player slots[MAX_PLAYERS];
13     int count;                 // giocatori attivi
14     int ready_count;           // giocatori pronti in lobby
15 } PlayerTable;
```

Listing 17: Struttura Player in player.h

Nota

Il socket del client **non** è un campo di `Player`: l'identità di un giocatore all'interno di `PlayerTable` è legata al `pthread_t` del thread che lo gestisce (`tid`), non al file descriptor. Il file descriptor è tenuto separatamente in due punti: come variabile locale `fd` nello stack di ciascun thread, e nell'array globale `int client_fds[MAX_PLAYERS]`, usato esclusivamente per il broadcast dei messaggi di lobby (`broadcast_lobby_update()`) verso i client ancora in attesa.

Assegnazione dello Slot

L'assegnazione avviene in `phase_join()`, che combina la ricerca di una cella di partenza libera nel labirinto con l'occupazione di uno slot in `PlayerTable`:

```
1 int phase_join(int fd, const char* nick) {
2     pthread_mutex_lock(&mutex);
3
4     if (maze.game_started) {
5         pthread_mutex_unlock(&mutex);
6         send_line(fd, "ERR game already in progress");
7         return -1;
8     }
9
10    int start_r, start_c;
11    if (!maze_random_free_cell(&maze, &start_r, &start_c)) {
12        pthread_mutex_unlock(&mutex);
13        send_line(fd, "ERR maze full");
14        return -1;
15    }
16
17    int idx = player_add(&pt, pthread_self(), nick, start_r,
18                        start_c);
19    if (idx >= 0) {
20        client_fds[idx] = fd;
21        broadcast_lobby_update();
22    }
23    pthread_mutex_unlock(&mutex);
24
25    if (idx < 0) send_line(fd, "ERR server full");
26    return idx;
27 }
```

Listing 18: Assegnazione slot e posizione iniziale in `server.c`

Nota

Un client che tenta di connettersi mentre `maze.game_started` è già vero viene respinto con `ERR game already in progress`: il sistema non supporta l'ingresso di nuovi giocatori a partita iniziata.

Nota

Il caso `ERR maze full` è un caso limite raro: si verifica solo se, per `MAX_PLAYERS` = 10 giocatori su un labirinto 20×20 (400 celle), non si trova più alcuna cella libera dopo `MAZE_ROWS*MAZE_COLS*2` tentativi casuali in `maze_random_free_cell()` — condizione praticamente impossibile dati i numeri in gioco, ma comunque gestita esplicitamente.

5.2.2 Struttura dei Mutex

Il server utilizza due mutex distinti per minimizzare la contesa:

```

1  /* Protegge maze e pt: tutto lo stato di gioco */
2  pthread_mutex_t mutex = PTHREAD_MUTEX_INITIALIZER;
3
4  /* Protegge l'accesso al file users.txt */
5  pthread_mutex_t auth_mutex = PTHREAD_MUTEX_INITIALIZER;

```

Listing 19: Dichiarazione dei mutex globali in server.c

Mutex	Risorsa protetta	Operazioni coperte
<code>mutex</code>	maze, pt	Mosse, aggiornamento punteggi, invio mappa globale, verifica fine partita
<code>auth_mutex</code>	File <code>users.txt</code>	Registrazione (REGISTER), autenticazione (LOGIN)

Nota

La separazione tra `mutex` e `auth_mutex` è una **scelta progettuale**: le operazioni di autenticazione (lettura e scrittura di `users.txt`) avvengono *prima* che il giocatore entri nel gioco.

Usare un unico mutex costringerebbe i thread già in partita ad attendere ogni nuovo accesso al file, aumentando inutilmente la latenza. **Con due mutex distinti, la fase di login non blocca mai lo stato di gioco, e viceversa.**

5.3 Meccanismo di Notifica tra Thread

5.3.1 Il problema: eventi esterni durante l'attesa in `select()`

Durante la fase di gioco, ciascun thread attende su `select()` sia dati dal proprio client sia lo scadere del timer della mappa globale. Tuttavia esiste un **terzo evento** che un thread deve poter rilevare immediatamente: la possibilità che **un altro** giocatore abbia raggiunto l'uscita o si sia disconnesso, cambiamento che può determinare la fine della partita per tutti. Senza un meccanismo dedicato, un thread se ne accorgerebbe solo al successivo timeout della mappa globale (fino a `GLOBAL_MAP_INTERVAL` secondi di ritardo).

5.3.2 La soluzione: un self-pipe condiviso

Il **self-pipe** è una tecnica che consente a un programma multi-threaded (o multiprocesso) di **notificare un evento tramite la memoria condivisa**. Al verificarsi dell'evento, si scrive un byte sulla pipe; monitorando l'estremità di lettura tramite una `select()` è possibile raccogliere questa notifica e agire di conseguenza, trattando l'evento come un normale caso di I/O.

Il server crea, una sola volta in `main()`, una pipe anonima globale:

```

1  int notify_pipe[2] = {-1, -1};
2  /* ... */
3  if (pipe(notify_pipe) < 0) { perror("pipe"); return 1; }
```

Listing 20: Creazione della pipe di notifica in `server.c`

Ogni thread, nel proprio ciclo di gioco, aggiunge `notify_pipe[0]` (estremità di lettura) all'insieme dei descrittori monitorati da `select()`, accanto al proprio socket, **nello stesso identico `select()` che gestisce anche il timeout della mappa globale** (si veda §6.4): non sono due meccanismi separati, ma un unico multiplexing su tre condizioni (socket, pipe, timeout):

```

1  fd_set rfd;
2  FD_ZERO(&rfd);
3  FD_SET(fd, &rfd);
4  FD_SET(notify_pipe[0], &rfd);
5  int maxfd = (fd > notify_pipe[0]) ? fd : notify_pipe[0];
6
7  int ready = select(maxfd + 1, &rfd, NULL, NULL, &tv);
8  /* ... */
9  if (FD_ISSET(notify_pipe[0], &rfd)) {
10     char buf[2];
11     int n = read(notify_pipe[0], buf, 2);
12     for (int i = 0; i < n; i++)
13         if (buf[i] == 0x01)
14             if (check_and_handle_game_end(fd, nick)) return;
15 }
```

Listing 21: `select()` su socket, pipe di notifica e timeout mappa globale (`server.c`)

Quando un client si disconnette, oppure quando un giocatore raggiunge l'uscita, la scrittura del byte sentinella nell'estremità di scrittura della pipe avviene così:

```
1 write(notify_pipe[1], "\x01", 1);
```

Listing 22: Scrittura del byte di notifica (server.c)

Poiché **tutti** i thread condividono la stessa estremità di lettura, questa scrittura sblocca istantaneamente ogni `select()` in attesa, inducendo ciascun thread a rivalutare `check_and_handle_game_end()` indipendentemente dal proprio timer residuo per la mappa globale.

Nota

In alcuni casi (come questo) è ragionevole usare una pipe come meccanismo di notifica generico all'interno di un `select()`, quando non è disponibile (o non si vuole usare) un meccanismo di sincronizzazione più specifico come le variabili di condizione POSIX (`pthread_cond_t`). Il vantaggio è che si integra naturalmente nello stesso `select()` già usato per il socket, senza dover gestire un secondo meccanismo di attesa in parallelo.

5.3.3 Il problema: sincronizzazione di fine partita

Quando un giocatore **trova l'uscita del labirinto**, oppure **quando scade il tempo massimo di partita**, il thread che se ne accorge deve inviare il messaggio `GAME_END` al proprio client e poi chiudere la connessione. Se ogni thread eseguisse questa operazione in autonomia, non appena rileva la fine della partita, si presenterebbe una

race condition:

un thread che si disconnette rimuove immediatamente la propria voce dalla tabella condivisa dei giocatori (`pt`, tramite `player_remove()`), rendendola invisibile alle elaborazioni di `game_check_end()` eseguiti dagli altri thread ancora in esecuzione. Il risultato osservabile è che **un giocatore uscito con successo può risultare "sparito" agli occhi degli altri client**, alterando la determinazione della vittoria/pareggio e lasciando alcuni client senza mai ricevere l'esito reale della partita.

È quindi necessario un punto di sincronizzazione che imponga la seguente regola:

- *nessun thread può disconnettersi finché **tutti i thread dei giocatori attivi non hanno constatato la fine della partita** e inviato il proprio `GAME_END`.*

5.3.4 La soluzione: le condition variable

La sincronizzazione è realizzata con una **condition variable POSIX** accoppiata a un contatore intero, entrambi protetti dal mutex globale già usato per l'accesso allo stato condiviso del gioco:

```
pthread_mutex_t mutex    = PTHREAD_MUTEX_INITIALIZER;
pthread_cond_t  all_end  = PTHREAD_COND_INITIALIZER;
int             all_done = 0;
```

- `all_done`
 - conta quanti thread hanno già rilevato la fine della partita e sono arrivati al punto di sincronizzazione;
 - viene azzerato da `client_cleanup()` quando l'ultimo giocatore si disconnette e il labirinto viene rigenerato, in modo da preparare correttamente il gioco la partita successiva.
- `all_end` è la condition variable su cui i thread si bloccano in attesa che il contatore raggiunga il numero totale di giocatori attivi (`pt.count`);

Le condition variable: Il meccanismo

La logica è contenuta in `check_and_handle_game_end()`, invocata periodicamente da ogni thread all'interno di `phase_play()`:

```
1
2 int check_and_handle_game_end(int fd, const char* nick) {
3     char winner[MAX_NICK_LEN];
4     int winner_score, is_draw;
5
6     pthread_mutex_lock(&mutex);
7     GameState status = game_check_end(&maze, &pt, winner,
8                                       &winner_score,
9                                       &is_draw);
10
11     if (status != GAME_RUNNING) {
12         char end_msg[MAX_MSG_LEN];
13         game_build_end_msg(winner, winner_score, is_draw,
14                           end_msg);
15         send_line(fd, end_msg);
16         log_write("GAME_END nick=%s msg=%s", nick, end_msg);
17
18         all_done++;
19         pthread_cond_broadcast(&all_end);
20         while (all_done < pt.count)
21             pthread_cond_wait(&all_end, &mutex);
22     }
23     pthread_mutex_unlock(&mutex);
24
25     if (status == GAME_RUNNING) return 0;
26     return 1;
27 }
```

Quando un thread, sotto mutex, rileva tramite `game_check_end()` che la partita è terminata (`status != GAME_RUNNING`):

1. costruisce e invia al proprio client il messaggio `GAME_END` con l'esito calcolato;
2. incrementa il contatore condiviso `all_done`;
3. notifica tutti gli altri thread eventualmente già in attesa, tramite `pthread_cond_broadcast(&all_end)`;
4. entra in un ciclo di attesa la cui condizione è la seguente: `while (all_done < pt.count) pthread_cond_wait(all_end, mutex);`.

Le condition variable: il ruolo

`pthread_cond_wait()` **rilascia atomicamente il mutex durante l'attesa** (evitando quindi un deadlock: se il mutex restasse acquisito, nessun altro thread potrebbe mai entrare nella sezione critica per incrementare a sua volta `all_done`) e quando viene **risvegliato**, rientra subito nella coda di attesa per **riottenerlo**, prima di rivalutare la condizione.

Il controllo avviene volutamente all'interno di un ciclo `while`, e non di un semplice `if`, secondo il pattern standard *wait-in-a-loop*: questo protegge sia da *spurious wakeup* (risvegli non associati a una reale variazione della condizione, ammessi dallo standard POSIX), sia dal caso in cui il thread si risvegli ma la condizione non sia ancora soddisfatta perché non tutti i giocatori hanno ancora raggiunto la barriera.

L'ultimo thread che incrementa `all_done` fino a farlo coincidere con `pt.count` fa uscire dal ciclo `while` anche tutti gli altri thread precedentemente bloccati: solo a quel punto ciascuno rilascia il mutex e ritorna da `check_and_handle_game_end()`, libero di proseguire verso la chiusura della connessione. In questo modo si garantisce che ogni giocatore abbia già ricevuto il proprio `GAME_END` coerente con lo stato finale condiviso, prima che una qualsiasi connessione venga chiusa e la relativa voce rimossa da `pt`.

Le condition variable: il motivo d'uso

Nel resto del server ogni thread deve restare *contemporaneamente* in ascolto su due sorgenti di eventi eterogenee:

- il socket del proprio giocatore (comandi `MOVE`, `QUIT`, ecc.)
- e i cambiamenti di stato prodotti dagli *altri* thread (ad esempio un altro giocatore che si disconnette).

Per questo motivo, ovunque tranne che nel blocco di fine partita, la sincronizzazione è realizzata con `select()` su un insieme di file descriptor, incluso l'espeditore del *self-pipe* (`notify_pipe`), che permette a un thread di "svegliare" il `select()` di un altro scrivendo un byte sulla pipe condivisa.

Una condition variable POSIX **non è multiplexabile con un file descriptor**: un thread bloccato in `pthread_cond_wait()` non può accorgersi, nello stesso momento, che sono arrivati dati sul socket del proprio client.

Nei punti in cui il thread deve restare reattivo sia all'input del proprio giocatore sia agli eventi generati da altri thread (`phase_play()`, `phase_wait_start()`), l'unico strumento compatibile con questa doppia esigenza è dunque `select()` con timeout e *self-pipe*, non una condition variable.

Il punto in cui viene usata la coppia `all_done/all_end` è **invece strutturalmente diverso**: nel momento in cui un thread rileva che la partita è terminata, non deve più leggere nulla dal socket del proprio client, avendo già ricevuto e processato l'ultimo comando rilevante (la mossa che ha causato l'uscita, o il rilevamento del timeout).

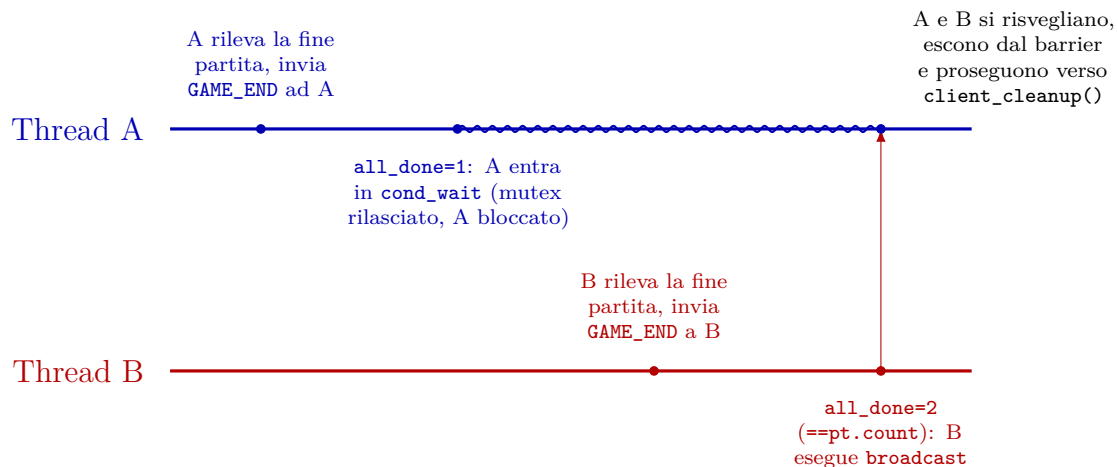
Il suo unico compito residuo è **attendere che anche i thread degli altri giocatori raggiungano lo stesso punto, per poi procedere tutti insieme**: un puro problema di *rendez-vous* fra thread, privo di qualsiasi componente di I/O da multiplexare — il caso d'uso canonico per cui esistono le condition variable.

In questo scenario specifico, **la condition variable è anche preferibile** alle alternative già impiegate altrove nel programma:

- rispetto a un'attesa attiva basata su polling con timeout (usata altrove per gestire l'I/O), non introduce latenza artificiale né consumo di CPU: il thread si risveglia esattamente quando l'ultimo giocatore raggiunge il punto di sincronizzazione.
- rispetto al solo mutex, che protegge l'accesso a `all_done` ma non offre di per sé un meccanismo di attesa bloccante su una condizione arbitraria, evita la necessità di un ciclo di busy-waiting sul lock, che sprecherebbe cicli di CPU.

Diagramma temporale

Il diagramma seguente mostra un caso concreto con due giocatori (A e B) che raggiungono la fine partita in istanti diversi: A arriva prima, si blocca in attesa; B arriva dopo, fa scattare la condizione di uscita e sblocca entrambi con un unico **broadcast**.



Si noti che, senza il punto di sincronizzazione, il thread A avrebbe potuto chiudere la connessione e rimuovere la propria voce da `pt` subito dopo il primo evento (non appena rileva la propria uscita dal labirinto), *prima* ancora che B avesse la possibilità di terminare la propria partita in modo coerente con la presenza di A nella tabella condivisa.

5.3.5 Schema riassuntivo: meccanismi di sincronizzazione tra thread

La tabella seguente riassume, per ciascun punto del server in cui thread diversi devono coordinarsi, quale meccanismo è stato scelto e perché:

Punto del server	Meccanismo	Cosa deve osservare il thread	Motivo della scelta
<code>phase_play()</code> (loop di gioco)	<code>select()</code> + self-pipe	Socket del proprio client e notifiche asincrone da altri thread	Due sorgenti di evento eterogenee (I/O di rete + segnalazioni inter-thread): serve multiplexing
<code>phase_wait_start()</code>	<code>select()</code> con timeout	Socket del proprio client, con controllo periodico di <code>maze.game_started</code>	Deve restare reattivo a un eventuale QUIT pur aspettando uno stato condiviso
<code>check_and_handle_game_end()</code> (barriera fine partita)	Condition variable	Solo lo stato condiviso: "tutti i thread hanno notificato la fine?"	Nessun I/O da multiplexare: puro rendez-vous fra thread

Tabella 4: Meccanismi di sincronizzazione usati nel server e relativo contesto d'uso.

6 Dettagli Implementativi

6.1 Definizioni Formali

Il presente progetto si prefigge l'obiettivo di **generare come campo da gioco un labirinto perfetto**:

Labirinto Perfetto

Un **labirinto perfetto** è un labirinto il cui grafo rappresentante $G = (V, E)$ delle celle percorribili è un **albero libero**.

Ossia:

- **Connessione:** Un grafo $G = (V, E)$ si definisce connesso $\iff$

$$\forall u, v \in V, \exists \pi(u, v)$$

cioè per ogni coppia di nodi esiste almeno un cammino che li collega.

- **Aciclicità:** Un grafo $G = (V, E)$ è detto **aciclico** se non contiene alcun percorso ciclico ossia:

$$\nexists \pi \in \text{Path}(G) : \text{first}(\pi) = \text{last}(\pi)$$

- tali proprietà derivano dalle proprietà di albero del grafo $G = (V, E)$
- **Assenza di radice intrinseca:**
 $\nexists$ nodo $r \in V$ tale che G sia dotato di una struttura gerarchica canonica indotta da r .

In particolare, un albero libero non è un grafo radicato cioè non è definita alcuna funzione di radicamento

$$f : V \rightarrow V$$

che assegni in modo intrinseco a ogni nodo un unico predecessore distinguendo un nodo radice.

da tale proprietà infatti deriva la seguente ed ultima:

- **Unicità del percorso:**

$$\forall u, v \in V, u \neq v : \exists! \pi(u, v)$$

cioè tra ogni coppia di nodi (celle del labirinto) esiste **un unico cammino semplice**.

Tali proprietà garantiscono che:

- esista esattamente un unico percorso tra qualsiasi coppia di celle del labirinto
- non vi siano cicli (percorsi alternativi)
- non vi siano celle irraggiungibili

Nota

Le proprietà dell'albero libero hanno ciascuna un significato intuitivo nel contesto del labirinto:

- connesso e aciclico significa che **si può raggiungere ogni cella senza mai tornare sulle proprie tracce**;
- $|E| = |V| - 1$ quantifica esattamente quanti muri devono essere **abbattuti** (termine definito in seguito);
- $\exists! \pi(u, v)$ è la proprietà di gioco che rende il **labirinto sfidante e univocamente risolvibile**.

6.1.1 Livello Logico: Struttura del Labirinto**Cella**

Una **cella** è l'unità atomica del labirinto. Formalmente è una coppia di indici logici (i, j) con $0 \leq i < N$ e $0 \leq j < M$, a cui è associato un **tipo** tramite la funzione di tipo τ :

$$\tau(i, j) \in T = \{ \text{WALL}, \text{FREE}, \text{EXIT}, \text{OBJECT} \}$$

Labirinto

Un **labirinto** è la coppia $\mathcal{M} = (\mathcal{G}, \tau)$ dove:

- $\mathcal{G} = \{(i, j) \mid 0 \leq i < N, 0 \leq j < M\}$ è l'insieme di tutte le celle, con $|\mathcal{G}| = N \cdot M$.
- $\tau : \mathcal{G} \rightarrow T$ è la funzione che assegna un tipo a ciascuna cella.

Le celle si partizionano nelle categorie disgiunte $\mathcal{G}_X = \tau^{-1}(X)$ per $X \in T$:

$$\mathcal{G} = \mathcal{G}_{\text{WALL}} \cup \mathcal{G}_{\text{FREE}} \cup \mathcal{G}_{\text{EXIT}} \cup \mathcal{G}_{\text{OBJ}} \quad (\text{unione disgiunta})$$

con i vincoli strutturali: $|\mathcal{G}_{\text{EXIT}}| = \text{NUM_EXITS} = 2$ e $|\mathcal{G}_{\text{OBJ}}| = \text{NUM_OBJECTS} = 10$.

Adiacenza tra Cella

Due celle (i, j) e (i', j') di $\mathcal{G}$ si dicono **adiacenti** ($\sim$) se differiscono di esattamente un'unità in una sola coordinata:

$$(i, j) \sim (i', j') \iff (|i - i'| = 1 \wedge j = j') \vee (i = i' \wedge |j - j'| = 1)$$

Le celle interne hanno esattamente 4 vicini (Nord, Sud, Est, Ovest); le celle di bordo ne hanno 2 o 3.

Grafo del Labirinto

Dato un labirinto $\mathcal{M} = (\mathcal{G}, \tau)$, il **grafo del labirinto** è $G = (V, E)$ dove:

$$V = \mathcal{G}_{\text{FREE}} \cup \mathcal{G}_{\text{EXIT}} \cup \mathcal{G}_{\text{OBJ}} \quad (\text{celle percorribili})$$

$$E = \{ \{u, v\} \mid u, v \in V, u \sim v \} \quad (\text{coppie adiacenti percorribili})$$

Un arco $\{u, v\} \in E$ esiste se e solo se entrambe le celle sono percorribili e adiacenti, ovvero il muro che le separa è assente.

6.1.2 Livello Fisico: Rappresentazione in Memoria

Modello a Griglia Fisica

Il labirinto $\mathcal{M}$ è rappresentato in memoria come una **griglia fisica** di dimensioni $(2N - 1) \times (2M - 1)$. Le posizioni nella griglia fisica si distinguono in due categorie in base alla parità degli indici:

Tipo	Indici fisici	Ruolo
Cella logica	$(2i, 2j)$	Nodo del grafo G
Muro interno	almeno un indice dispari	Arco potenziale di G

Muro

Un **muro** è una **cella della griglia fisica a indici dispari** che separa due celle logiche adiacenti.

Definizione

Si distinguono due tipi di muri:

- **Muro orizzontale** $\mathcal{W}_H$: separa (i, j) e $(i+1, j)$, si trova all'indice fisico $(2i+1, 2j)$ con $0 \leq i < N - 1, 0 \leq j < M$.
- **Muro verticale** $\mathcal{W}_V$: separa (i, j) e $(i, j+1)$, si trova all'indice fisico $(2i, 2j+1)$ con $0 \leq i < N, 0 \leq j < M - 1$.

$$\mathcal{W} = \mathcal{W}_H \cup \mathcal{W}_V \quad |\mathcal{W}| = (N - 1) \cdot M + N \cdot (M - 1)$$

Definizione

Un muro $w \in \mathcal{W}$ è **abbattuto** se $\tau(w) = \text{FREE}$, **integro** se $\tau(w) = \text{WALL}$. Il collegamento con il grafo astratto è diretto:

$$\{u, v\} \in E \iff w_{uv} \in \mathcal{W} \wedge \tau(w_{uv}) = \text{FREE}$$

Nota

Il numero di muri abbattuti al termine della generazione coincide esattamente con $|E|$. Per la proprietà di albero, in un labirinto perfetto con $N \cdot M = 100$ celle logiche devono essere abbattuti esattamente $|V| - 1 = 99$ muri su $|\mathcal{W}| = 180$ disponibili. I restanti $180 - 99 = 81$ muri rimangono integri, definendo la struttura dei corridoi.

6.2 Generazione del Labirinto

6.2.1 Algoritmo di Prim

L'algoritmo di Prim randomizzato garantisce la proprietà di labirinto perfetto *per costruzione*: ad ogni iterazione un muro abbattuto connette una nuova cella $v \notin \mathcal{L}$ all'insieme $\mathcal{L}$ delle celle già liberate, aggiungendo esattamente un arco a E senza mai chiudere un ciclo.

6.2.2 Fondamenti Teorici

L'algoritmo di Prim nasce originariamente come algoritmo per la costruzione di un **albero di copertura minimo** (Minimum Spanning Tree, MST) su un grafo pesato. La sua versione *randomizzata* applicata ai labirinti sfrutta la stessa struttura logica, ma sostituisce la priorità dei pesi con una selezione *casuale uniforme*.

6.2.3 Descrizione dell'Algoritmo

Il principio di funzionamento può essere riassunto in tre fasi concettuali:

Fase 1 — Inizializzazione. La griglia viene riempita interamente di muri. Si sceglie una cella seme (nel progetto la cella logica $(0, 0)$, corrispondente all'indice fisico $(0, 0)$). Questa cella diventa *libera* e tutti i muri che la circondano (al massimo quattro, uno per ogni direzione cardinale) vengono aggiunti a una struttura dati chiamata **frontiera**.

Fase 2 — Crescita iterativa. Finché la frontiera non è vuota, si estrae un muro *a caso*. Un muro della frontiera separa una cella già visitata da una cella ancora non ancora visitata. Una volta scelto un muro randomicamente, se la cella comunicante con quest'ultimo non è stata ancora visitata, possono essere compiute due azioni: si *abbatte il muro intermedio* che separa le due celle, aggiungendo a sua volta i suoi muri intatti alla frontiera. Se invece la cella destinazione è già libera, il muro viene semplicemente scartato: includerlo creerebbe un ciclo, violando la proprietà di labirinto perfetto.

Fase 3 — Decorazione. Al termine della generazione tutte le celle logiche sono state raggiunte, formando un albero (nessun ciclo, tutto connesso). Si posizionano poi `NUM_EXITS` uscite e `NUM_OBJECTS` oggetti raccogliibili in celle libere scelte casualmente.

6.2.4 Pseudocodice

Algorithm 1 Generazione Labirinto con Prim Randomizzato

```

1: procedura GENERALABIRINTO(griglia  $L$  di dimensioni  $N \times M$ )
2:   // Fase 1: inizializzazione
3:   for each  $(i, j)$  in  $G$  do
4:      $L[i][j] \leftarrow \text{WALL}$ 
5:   end for
6:   Scegli la cella seme  $s \leftarrow (0, 0)$ 
7:    $L[s] \leftarrow \text{VISITED}$ 
8:    $F \leftarrow \emptyset$  ▷ Frontiera inizialmente vuota
9:   for each direction  $d \in \{N, S, E, O\}$  do
10:     $F \leftarrow F \cup \{w\}$ 
11:  end for
12:  // Fase 2: crescita iterativa
13:  while  $F \neq \emptyset$  do
14:    Estrai un muro  $w = (r_w, c_w, d)$  a caso da  $F$ 
15:     $\text{dest} \leftarrow \text{cella\_destinazione}(w)$  ▷ 2 passi nella direzione  $d$ 
16:    if  $G[\text{dest}] = \text{MURO}$  then ▷ La cella non è ancora stata visitata
17:       $G[\text{muro\_intermedio}(w)] \leftarrow \text{LIBERA}$  ▷ Abbatti il muro di separazione
18:       $G[\text{dest}] \leftarrow \text{LIBERA}$ 
19:      for ogni direzione  $d' \in \{N, S, E, O\}$  do
20:        if il muro  $w' = \text{muro}(\text{dest}, d')$  è dentro i bordi then
21:           $F \leftarrow F \cup \{w'\}$ 
22:        end if
23:      end for
24:    end if ▷ Se  $\text{dest}$  era già libera, il muro è scartato (evita cicli)
25:  end while
26:  // Fase 3: decorazione
27:  Posiziona NUM_EXITS uscite in celle libere casuali
28:  Posiziona NUM_OBJECTS oggetti in celle libere casuali
29: end procedura

```

Nota

La scelta *casuale* del muro da estrarre alla riga 11 è il punto cruciale che distingue l'algoritmo di Prim randomizzato dall'originale algoritmo di Prim deterministico (che estrarrebbe il muro di peso minimo). La casualità garantisce che i percorsi si ramifichino in modo uniforme su tutta la griglia, producendo labirinti visivamente equilibrati anziché concentrare la crescita in una sola direzione.

6.2.5 La Frontiera: Definizione Formale e Ruolo

Frontiera dell'Algoritmo di Prim

Dato l'insieme $\mathcal{L}$ delle celle già *liberate* durante l'esecuzione dell'algoritmo, la **frontiera** F è l'insieme di tutti i *muri intermedi* che soddisfano le seguenti tre condizioni simultaneamente:

1. Il muro si trova fisicamente agli indici *dispari* della griglia (è un muro di separazione, non una cella logica).
2. Sul lato "interno" del muro (nella direzione verso $\mathcal{L}$) si trova una cella già liberata: $\text{src} \in \mathcal{L}$.
3. Sul lato "esterno" del muro si trova una cella ancora murata: $\text{dest} \notin \mathcal{L}$.

Formalmente:

$$F = \left\{ w \mid w = \text{muro}(\text{src}, d), \text{src} \in \mathcal{L}, \text{dest}(w) \notin \mathcal{L}, \text{dest}(w) \in \text{bordi validi} \right\}$$

dove $\text{dest}(w)$ indica la cella logica raggiungibile abbattendo il muro w .

Invariante di frontiera. In ogni momento dell'esecuzione vale l'invariante:

$$F \neq \emptyset \iff \mathcal{L} \neq (\text{tutte le celle logiche})$$

ossia la frontiera è vuota *se e solo se* tutte le celle logiche sono state raggiunte e il labirinto è completo. Questa proprietà garantisce la terminazione dell'algoritmo.

Nota

Perché alcuni muri in F vengono scartati? Può accadere che, al momento dell'estrazione di un muro w , la sua cella destinazione $\text{dest}(w)$ sia *già libera*: questo significa che è stata raggiunta tramite un altro percorso *dopo* che w era stato inserito in F . Abbattere w in questo caso creerebbe un *ciclo*, violando la proprietà di labirinto perfetto. Il muro viene quindi scartato senza effettuare alcuna modifica alla griglia.

6.2.6

La Frontiera: Implementazione

In `maze.c` la **frontiera** è implementata tramite una lista dinamica.

Definizione

Ogni elemento della frontiera è una terna (r, c, d) che identifica il muro:

```

1 typedef struct { int r, c, dir; } Wall;
2
3 static Wall *frontier = NULL;
4 static int frontier_sz = 0;
5 static int frontier_cap = 0;
6
7 static void frontier_push(int r, int c, int dir) {
8     if (frontier_sz == frontier_cap) {
9         frontier_cap = frontier_cap ? frontier_cap * 2 : 64;
10        frontier = realloc(frontier, frontier_cap *
11                           sizeof(Wall));
12    }
13    frontier[frontier_sz++] = (Wall){r, c, dir};
14 }
```

Listing 23: Struttura della frontiera in `maze.c`

L'estrazione casuale avviene con il *pop random swap-with-last*, che mantiene la complessità $O(1)$ per ogni estrazione:

```

1 static Wall frontier_pop_random(void) {
2     int idx = rand() % frontier_sz;
3     Wall w = frontier[idx];
4     frontier[idx] = frontier[--frontier_sz]; /* swap con
5        l'ultimo */
6     return w;
7 }
```

Listing 24: Pop casuale dalla frontiera — $O(1)$

Nota

La dimensione massima della frontiera è proporzionale al perimetro di $\mathcal{L}$ e non supera mai $O(N \cdot M)$. In pratica, per la griglia 20×20 del progetto, la frontiera contiene al massimo qualche centinaio di elementi: il costo di memoria è del tutto trascurabile.

Esempio

Si consideri una griglia 5×5 (logica 3×3 , per semplicità) all'inizio dell'algoritmo. Dopo la liberazione della cella seme (0,0):

```

      L  W  .  .  .
      W  .  .  .  .
      .  .  .  .  .
      .  .  .  .  .
      .  .  .  .  .

```

dove L = cella liberata, W = muro in frontiera, . = muro non ancora in frontiera. La frontiera F contiene esattamente i due muri W adiacenti alla cella seme: quello a Est (0,1) e quello a Sud (1,0). Al passo successivo, viene estratto uno dei due a caso.

```

1 void maze_generate(Maze *maze) {
2     /* 1. riempie tutto di muri */
3     for (int r = 0; r < MAZE_ROWS; r++)
4         for (int c = 0; c < MAZE_COLS; c++)
5             maze->grid[r][c].cell = CELL_WALL;
6
7     /* 2. cella seme (0,0) */
8     int sr = 0, sc = 0;
9     maze->grid[sr][sc].cell = CELL_FREE;
10    add_frontier(maze, sr, sc);
11
12    /* 3. ciclo principale */
13    while (frontier_sz > 0) {
14        Wall w = frontier_pop_random();
15        int nr = w.r + DR2[w.dir];
16        int nc = w.c + DC2[w.dir];
17        if (nr < 0 || nr >= MAZE_ROWS || nc < 0 || nc >=
18            MAZE_COLS)
19            continue;
20        if (maze->grid[nr][nc].cell != CELL_WALL)
21            continue; /* gia' libera: scarta il muro
22                       (evita cicli) */
23
24        /* abbatti il muro intermedio */
25        maze->grid[w.r + DR1[w.dir]][w.c + DC1[w.dir]].cell
26            = CELL_FREE;
27        /* libera la destinazione */
28        maze->grid[nr][nc].cell = CELL_FREE;
29        add_frontier(maze, nr, nc);
30    }
31    free(frontier);
32    frontier = NULL; frontier_sz = frontier_cap = 0;

```

Listing 25: Funzione maze_generate in maze.c

```
1
2      /* 4. posiziona NUM_EXITS uscite in celle libere casuali */
3      int placed = 0;
4      while (placed < NUM_EXITS) {
5          int r = rand() % MAZE_ROWS, c = rand() % MAZE_COLS;
6          if (maze->grid[r][c].cell == CELL_FREE) {
7              maze->grid[r][c].cell = CELL_EXIT;
8              placed++;
9          }
10     }
11
12     /* 5. posiziona NUM_OBJECTS oggetti in celle libere casuali */
13     placed = 0;
14     while (placed < NUM_OBJECTS) {
15         int r = rand() % MAZE_ROWS, c = rand() % MAZE_COLS;
16         if (maze->grid[r][c].cell == CELL_FREE) {
17             maze->grid[r][c].cell = CELL_OBJECT;
18             placed++;
19         }
20     }
21
22     maze->num_objects = NUM_OBJECTS;
23     maze->num_exits = NUM_EXITS;
24     maze->game_started = 0;
25 }
```

6.3 Mappa Locale e Mappa Globale

6.3.1 Differenza Concettuale

Proprietà	Mappa Locale	Mappa Globale
Dimensione	3×3 (VIEW_RADIUS=1)	20×20 (intera)
Trigger	Dopo ogni MOVE	Ogni T secondi (periodica)
Celle non viste	Non compaiono	Mostrate come ?
Scopo	Navigazione immediata	Visione d'insieme

6.3.2 Costruzione della Mappa Locale

La mappa locale è una *finestra di osservazione* centrata sulla posizione corrente del giocatore (p_{row}, p_{col}). La finestra ha lato $2V + 1$ dove $V = \text{VIEW_RADIUS} = 1$, quindi misura $3 \times 3 = 9$ celle.

Processo di costruzione La funzione `player_local_map()` itera su tutti gli spostamenti $(\Delta r, \Delta c)$ con $\Delta r, \Delta c \in \{-1, 0, 1\}$, producendo in ordine riga-per-riga le 9 celle della finestra. Per ogni cella $(r, c) = (p_{row} + \Delta r, p_{col} + \Delta c)$ si applicano le seguenti regole in cascata:

- Regola 1. Fuori dai bordi:** se (r, c) cade fuori dalla griglia ($r < 0$ oppure $r \geq \text{MAZE_ROWS}$, idem per c), la cella viene rappresentata come **WALL**. Questo garantisce che il labirinto appaia “circondato da muri” ai bordi, senza accessi speciali fuori area.
- Regola 2. Posizione del giocatore:** se $\Delta r = 0$ e $\Delta c = 0$, ovvero ci troviamo esattamente sulla cella occupata dal giocatore, il carattere inserito è **PLAYER**. Il simbolo del giocatore sovrascrive qualsiasi contenuto sottostante (cella libera, oggetto, uscita).
- Regola 3. Cella non ancora scoperta:** se la cella (r, c) è all'interno della griglia ma il campo `discovered[r][c]` del giocatore è 0 (cella mai esplorata), viene inserito il carattere **UNKNOWN** (?). La mappa locale mostra quindi solo ciò che il giocatore ha effettivamente visto.
- Regola 4. Cella scoperta:** se nessuna delle regole precedenti si applica, si inserisce il contenuto reale della cella `maze->grid[r][c].cell`, che può essere **WALL**, **FREE**, **EXIT** o **OBJECT**.

Ordine di priorità. Le regole sono applicate in cascata con un `if-else if-else`: la priorità è *bordo* > *giocatore* > *sconosciuto* > *reale*. Questo ordine è importante: senza la regola 2 il giocatore non sarebbe mai visibile (la sua cella sarebbe mostrata come contenuto reale); senza la regola 1 si accederebbero indirizzi fuori array.

Il buffer di output. Il risultato è una stringa di $3 \times 3 = 9$ caratteri ASCII (più il terminatore `\0`), serializzata riga per riga da sinistra a destra. Questa stringa è quella trasmessa nel messaggio `LOCAL 3 3 <data>`. Il client la decodifica posizione per posizione, applicando i colori ANSI e i simboli Unicode corrispondenti.

```

1 void player_local_map(const Player *p, const Maze* maze,
2   char *buf) {
3   int idx = 0;
4   for (int r_to_see = -VIEW_RADIUS; r_to_see <=
5     VIEW_RADIUS; r_to_see++) {
6     for (int c_to_see = -VIEW_RADIUS; c_to_see <=
7       VIEW_RADIUS; c_to_see++) {
8       int r = p->row + r_to_see, c = p->col + c_to_see;
9       char ch;
10      /* Regola 1: fuori dai bordi */
11      if (r < 0 || r >= MAZE_ROWS || c < 0 || c >=
12        MAZE_COLS)
13        ch = CELL_WALL;
14      /* Regola 2: posizione del giocatore */
15      else if (r_to_see == 0 && c_to_see == 0)
16        ch = CELL_PLAYER;
17      /* Regola 3: cella non scoperta */
18      else if (!p->discovered[r][c])
19        ch = CELL_UNKNOWN;
20      /* Regola 4: contenuto reale */
21      else ch = maze->grid[r][c].cell;
22      buf[idx++] = ch;
23    }
24  }
25  buf[idx] = '\0';
26 }

```

Listing 26: Costruzione mappa locale in player.c

Esempio

Supponiamo che il giocatore si trovi in posizione (10,10) e che nell'area attorno a lui il labirinto contenga un muro a Nord e un oggetto a Est. La finestra 3×3 potrebbe apparire come (simboli semplificati):

```

+ - - - +
| # # # |
| . P @ |
| # . . |
+ - - - +

```

Dove P è la posizione corrente, @ è l'oggetto raccoglibile, # sono i muri scoperti e . le celle libere già viste.

6.3.3 Rivelazione delle Celle

Ogni volta che il giocatore si muove, `player_reveal()` marca come scoperte tutte le celle nel quadrato di lato $2V + 1$ centrato sulla nuova posizione:

```
1 void player_reveal(Player *p, int row, int col) {  
2     for (int r_to_see = -VIEW_RADIUS; r_to_see <=  
3         VIEW_RADIUS; r_to_see++) {  
4         for (int c_to_see = -VIEW_RADIUS; c_to_see <=  
5             VIEW_RADIUS; c_to_see++) {  
6                 int r = row + r_to_see;  
7                 int c = col + c_to_see;  
8                 if (r >= 0 && r < MAZE_ROWS && c >= 0 && c <  
9                     MAZE_COLS)  
10                    p->discovered[r][c] = 1;  
11             }  
12     }  
13 }
```

Listing 27: Rivelazione delle celle in `player.c`

Con `VIEW_RADIUS = 1`, ogni mossa rivela al massimo $3 \times 3 = 9$ celle.

Nota

Data la *natura* dell'algoritmo di Prim utilizzato per generare il labirinto, non avrebbe senso mostrare una mappa locale con **raggio** > 1 . Questo perché l'algoritmo crea sempre dei corridoi larghi una casella, quindi le celle in più che verrebbero considerate da un raggio maggiore sarebbero sempre **oscurate**.

6.4 Invio Periodico della Mappa Globale: Il Meccanismo del Timeout

6.4.1 Il Problema: Tre Sorgenti di Input Asincrone

Ogni thread deve gestire *contemporaneamente* tre flussi di eventi indipendenti, tutti multiplexati in un unico `select()` (si veda anche §5.3 per il self-pipe):

1. I **comandi del client** che arrivano sul socket, in qualsiasi momento.
2. Le **notifiche asincrone** da altri thread tramite il self-pipe `notify_pipe`, ad esempio quando un altro giocatore esce o si disconnette.
3. La **scadenza periodica del timer** (ogni `GLOBAL_MAP_INTERVAL` secondi), che richiede di inviare proattivamente la mappa globale senza alcuna richiesta esplicita dal client.

Una lettura bloccante (`read()`) su un solo socket impedirebbe di rilevare sia il self-pipe sia la scadenza del timer. La soluzione adottata è il multiplexing I/O con `select()` su *entrambi* i file descriptor (socket e pipe) e un **timeout variabile**.

Il loop di gioco calcola ad ogni iterazione *quanto tempo manca* al prossimo invio della mappa globale, e passa questo valore come timeout a `select()`. `select()` attende al massimo quel numero di secondi; può sbloccarsi prima se arrivano dati sul socket o sulla pipe di notifica. In base a ciò che ha causato il ritorno di `select()`, il thread esegue l'azione appropriata:

- **Ritorno per dati sul socket** (`FD_ISSET(fd, &rdfs)`): è arrivato un comando dal client. Il thread legge e processa il comando normalmente. *Il timer non viene resettato*: la variabile `last_global` non viene modificata, quindi al prossimo ciclo il timeout sarà più breve, proporzionalmente al tempo già trascorso.
- **Ritorno per notifica sulla pipe** (`FD_ISSET(notify_pipe[0], &rdfs)`): un altro thread ha segnalato un evento (uscita o disconnessione di un giocatore). Il thread legge il byte sentinella e richiama `check_and_handle_game_end()` per rivalutare subito l'esito della partita.
- **Ritorno per timeout** (`select()` restituisce 0): il tempo massimo è scaduto senza che arrivassero dati. Si acquisisce il mutex, si costruisce e invia la mappa, si rilascia il mutex, si aggiorna `last_global` e si richiama nuovamente `check_and_handle_game_end()` (per rilevare un eventuale timeout di partita).

Perché il timeout è *variabile* e non fisso. Il timeout passato a `select()` viene ricalcolato a ogni iterazione come:

$$\text{remaining} = \text{GLOBAL_MAP_INTERVAL} - (t_{\text{now}} - t_{\text{last}})$$

Questo è necessario perché, se al ciclo precedente il client ha inviato molti comandi in rapida successione, il tempo effettivo già consumato potrebbe essere molto vicino all'intervallo desiderato. Usando un timeout fisso si rischierebbe di ritardare sistematicamente l'invio. Con il timeout variabile, il meccanismo rimane preciso indipendentemente dall'attività del client.

Gestione del caso $\text{remaining} \leq 0$. Se al momento del calcolo il tempo trascorso supera già l'intervallo ($\text{remaining} \leq 0$), il timeout viene forzato a zero: `select()` ritorna immediatamente, attivando l'invio della mappa senza ulteriori attese.

```

1 void phase_play(int fd, int player_idx, const char* nick) {
2     send_line(fd, "START");
3
4     pthread_mutex_lock(&mutex);
5     client_fds[player_idx] = -1; /* esce dai destinatari del
6         broadcast lobby */
7     send_local_map(fd, &pt.slots[player_idx]);
8     pthread_mutex_unlock(&mutex);
9     time_t last_global = time(NULL);
10
11     while (1) {
12         if (check_and_handle_game_end(fd, nick)) return;
13         /* --- calcolo del timeout variabile --- */
14         time_t now = time(NULL);
15         long elapsed = (long)(now - last_global);
16         long remaining = (long)GLOBAL_MAP_INTERVAL -
17             elapsed;
18         if (remaining <= 0) remaining = 0; /* evita
19             timeout negativi */
20
21         struct timeval tv = { .tv_sec = remaining, .tv_usec
22             = 0 };
23         fd_set rfd;
24         FD_ZERO(&rfd);
25         FD_SET(fd, &rfd);
26         FD_SET(notify_pipe[0], &rfd);
27         int maxfd = (fd > notify_pipe[0]) ? fd :
28             notify_pipe[0];
29
30         int ready = select(maxfd + 1, &rfd, NULL, NULL,
31             &tv);
32         if (ready < 0) {
33             if (errno == EINTR) continue;
34             perror("select");
35             return;
36         }
37         if (ready == 0) {
38             /* --- TIMEOUT: invia mappa globale --- */
39             pthread_mutex_lock(&mutex);
40             send_global_map(fd, &pt.slots[player_idx]);
41             pthread_mutex_unlock(&mutex);
42             last_global = time(NULL); /* resetta il
43                 riferimento temporale */
44             if (check_and_handle_game_end(fd, nick)) return;
45             continue;
46         }
47     }
48 }

```

```

42     if (FD_ISSET(notify_pipe[0], &rfds)) {
43         /* --- NOTIFICA: un altro thread ha segnalato un
44            evento --- */
45         char buf[2];
46         int n = read(notify_pipe[0], buf, 2);
47         for (int i = 0; i < n; i++)
48             if (buf[i] == 0x01)
49                 if (check_and_handle_game_end(fd, nick))
50                     return;
51     }
52
53     if (FD_ISSET(fd, &rfds)) {
54         /* --- DATI: processa il comando del client ---
55            */
56         int result = handle_play_command(fd, player_idx,
57            nick);
58         if (result == 0) return; /*
59            disconnesso o QUIT */
60         if (result == 2) /*
61            uscito dal labirinto */
62             if (check_and_handle_game_end(fd, nick))
63                 return;
64     }
65 }

```

Listing 28: Loop di gioco completo, con `select()` su socket+pipe e timer variabile (`server.c`)

Nota

Questa soluzione è **autonoma per ogni thread**: ogni thread gestisce il proprio timer indipendentemente dagli altri, senza comunicazione con il thread principale. Due thread con diversi momenti di connessione invieranno le mappe globali in momenti sfasati, il che è corretto: ogni giocatore riceve la *propria* mappa personale (basata sulla sua matrice `discovered`) al momento giusto.

Nota

A differenza del modello multi-processo, non è necessario gestire il segnale `SIGCHLD`. La segnalazione `SIGPIPE` viene ignorata nel `main()` tramite `signal(SIGPIPE, SIG_IGN)`: questo evita che il processo termini quando un thread tenta di scrivere su un socket già chiuso dal client.

6.5 Meccanica di Punteggio e Interazione con la Griglia

6.5.1 Movimento e Validazione

Ogni **MOVE** calcola la nuova posizione e la convalida *prima* di applicarla, sotto la protezione di `mutex`:

```
1  switch (dir) {
2      case 'N': new_r--; break;
3      case 'S': new_r++; break;
4      case 'W': new_c--; break;
5      case 'E': new_c++; break;
6      default:
7          pthread_mutex_unlock(&mutex);
8          send_line(fd, "ERR unknown direction");
9          return 1;
10 }
11
12 if (new_r < 0 || new_r >= MAZE_ROWS || new_c < 0 || new_c >=
13     MAZE_COLS ||
14     maze.grid[new_r][new_c].cell == CELL_WALL) {
15     pthread_mutex_unlock(&mutex);
16     send_line(fd, "ERR wall");
17     return 1;
18 }
19 p->row = new_r; p->col = new_c;
20 player_reveal(p, new_r, new_c);
```

Listing 29: Validazione del movimento in `handle_move_command` (`server.c`)

6.5.2 Raccolta degli Oggetti

Se la cella di destinazione contiene un oggetto, `maze_collect_object()` lo rimuove atomicamente dalla griglia condivisa e il punteggio del giocatore viene incrementato di **un punto** (non un valore configurabile):

```

1 if (maze_collect_object(&maze, new_r, new_c)) {
2     p->score++;
3     char msg[64];
4     snprintf(msg, sizeof(msg), "COLLECT %d", p->score);
5     send_line(fd, msg);
6     log_write("COLLECT nick=%s score=%d pos=(%d,%d)", nick,
7             p->score, new_r, new_c);
8 }
```

Listing 30: Raccolta oggetto in `handle_move_command` (server.c)

```

1 int maze_collect_object(Maze *maze, int row, int col) {
2     if (maze->grid[row][col].cell == CELL_OBJECT) {
3         maze->grid[row][col].cell = CELL_FREE;
4         maze->num_objects--;
5         return 1;
6     }
7     return 0;
8 }
```

Listing 31: Rimozione dell'oggetto dalla griglia (maze.c)

Nota

La rimozione agisce sulla griglia **condivisa** `maze.grid`: un oggetto raccolto da un giocatore scompare per tutti. Il campo `<score>` nel messaggio `COLLECT <score>` è il punteggio *totale* del giocatore dopo l'incremento, non il solo punto guadagnato.

6.6 Raggiungimento di un'uscita

```

1 if (maze.grid[new_r][new_c].cell == CELL_EXIT) {
2     p->exited = 1;
3     char msg[64];
4     snprintf(msg, sizeof(msg), "EXIT_OK %d", p->score);
5     send_line(fd, msg);
6     log_write("EXIT nick=%s score=%d", nick, p->score);
7     write(notify_pipe[1], "\x01", 1);
8     pthread_mutex_unlock(&mutex);
9     return 2; /* segnala a phase_play che il giocatore e'
10             uscito */
11 }
```

Listing 32: Gestione dell'uscita in `handle_move_command` (server.c)

Il valore di ritorno 2 risale fino a `phase_play()`, che invoca immediatamente `check_and_handle_game_end()` anziché terminare subito il thread: il giocatore uscito resta connesso e riceve comunque il `GAME_END` finale, come già osservato in §5.3 a

proposito della sveglia degli altri thread. Si noti inoltre che l'uscita di un giocatore scrive essa stessa un byte sul `notify_pipe`, così da svegliare immediatamente anche gli altri thread ancora in attesa, senza dover aspettare il loro prossimo timeout.

6.7 Logica di Fine Partita: Implementazione della Verifica

6.7.1 Funzionamento informale della verifica

La funzione `game_check_end()` viene chiamata in tre punti critici del loop di gioco (si veda §6.4): all'inizio di ogni iterazione (per rilevare un esito già maturato), dopo la scadenza del timeout della mappa globale, e dopo ogni mossa in cui il giocatore raggiunge l'uscita. Il meccanismo opera come segue:

Passo 0 — Caso limite: un solo giocatore rimasto. Se nella `PlayerTable` è rimasto un solo slot attivo (`pt->count == 1`), tipicamente perché tutti gli altri giocatori si sono disconnessi, quell'unico giocatore rimasto viene dichiarato vincitore immediato con il proprio punteggio corrente, senza attendere né l'uscita dal labirinto né il timeout.

Passo 1 — Verifica del timeout. Si calcola il tempo trascorso dall'avvio della partita usando `time(NULL)` e il campo `maze->start_time`. Se la differenza supera `GAME_TIMEOUT` secondi, la variabile `timed_out` viene posta a `true`.

Passo 2 — Scansione della tabella giocatori. Si scorre l'intero array `PlayerTable` (tutti i `MAX_PLAYERS` slot). Per ogni slot attivo (`active == 1`) si tiene traccia *contemporaneamente* di due classifiche distinte:

- il punteggio migliore tra **tutti** i giocatori attivi (usato solo se, al momento del timeout, nessuno è ancora uscito);
- il punteggio migliore tra i soli **giocatori usciti** (`exited == 1`), insieme al conteggio `exited_count` e al conteggio dei giocatori ancora in partita `ingame_count`. Se due o più giocatori usciti condividono il punteggio massimo, si imposta il flag di pareggio corrispondente.

Passo 3 — Determinazione dell'esito. Le condizioni vengono valutate nell'ordine:

1. Se `ingame_count == 0` e `exited_count >= 1` (tutti i giocatori ancora attivi sono usciti), la partita termina subito: vince l'uscito con il punteggio più alto; se erano usciti in due o più con lo stesso punteggio massimo, è pareggio.
2. Se il timeout è scaduto (`timed_out`), l'esito dipende da quanti giocatori sono usciti: con un solo uscito vince lui; con due o più usciti vince il migliore tra loro (con eventuale pareggio); se nessuno è ancora uscito, vince il miglior punteggio tra *tutti* i giocatori attivi (con eventuale pareggio).
3. Altrimenti: la partita è ancora in corso (`GAME_RUNNING`).

Propagazione del risultato. Una volta determinato l'esito, la funzione `game_build_end_msg()` costruisce la stringa del protocollo (`GAME_END WIN` o `GAME_END DRAW`) a partire da nickname, punteggio e flag di pareggio già calcolati. Il messaggio viene poi inviato al client tramite `send_line()`.

6.7.2 Implementazione

```

1  GameState game_check_end(const Maze *maze, const
    PlayerTable *pt, char *winner_nick, int *winner_score,
    int *draw) {
2      *winner_nick = '\0';
3      *winner_score = 0;
4      *draw = 0;
5
6      /* Passo 0: un solo giocatore rimasto -> vince lui
        immediatamente */
7      if (pt->count == 1) {
8          for (int i = 0; i < MAX_PLAYERS; i++) {
9              if (pt->slots[i].active) {
10                 strncpy(winner_nick, pt->slots[i].nick,
                     MAX_NICK_LEN - 1);
11                 *winner_score = pt->slots[i].score;
12                 *draw = 0;
13                 return GAME_OVER_EXIT;
14             }
15         }
16     }
17
18     /* Passo 1: verifica timeout */
19     int timed_out = (time(NULL) - maze->start_time) >=
        GAME_TIMEOUT;
20
21     /* Passo 2: scansione degli slot attivi, doppia
        classifica */
22     int exited_count = 0, ingame_count = 0;
23     int best_exited_score = -1, exited_tied = 0;
24     char best_exited_nick[MAX_NICK_LEN] = {0};
25     int best_any_score = -1, any_tied = 0;
26     char best_any_nick[MAX_NICK_LEN] = {0};
27
28     for (int i = 0; i < MAX_PLAYERS; i++) {
29         if (!pt->slots[i].active) continue;
30
31         if (pt->slots[i].score > best_any_score) {
32             best_any_score = pt->slots[i].score;
33             strncpy(best_any_nick, pt->slots[i].nick,
                     MAX_NICK_LEN - 1);
34             any_tied = 0;
35         } else if (pt->slots[i].score == best_any_score) {
36             any_tied = 1;
37         }
38
39         if (pt->slots[i].exited) {
40             exited_count++;
41             if (pt->slots[i].score > best_exited_score) {
42                 best_exited_score = pt->slots[i].score;

```

```

43         strncpy(best_exited_nick, pt->slots[i].nick,
44                 MAX_NICK_LEN - 1);
45         exited_tied = 0;
46     } else if (pt->slots[i].score ==
47               best_exited_score) {
48         exited_tied = 1;
49     }
50 } else {
51     ingame_count++;
52 }
53
54 /* Passo 3: determinazione esito */
55
56 /* tutti i giocatori ancora attivi sono usciti */
57 if (ingame_count == 0 && exited_count >= 1) {
58     strncpy(winner_nick, best_exited_nick, MAX_NICK_LEN
59           - 1);
60     *winner_score = best_exited_score;
61     *draw = (exited_count == 1) ? 0 : exited_tied;
62     return GAME_OVER_EXIT;
63 }
64
65 /* timeout scaduto */
66 if (timed_out) {
67     if (exited_count == 1) {
68         strncpy(winner_nick, best_exited_nick,
69               MAX_NICK_LEN - 1);
70         *winner_score = best_exited_score;
71         *draw = 0;
72     } else if (exited_count >= 2) {
73         strncpy(winner_nick, best_exited_nick,
74               MAX_NICK_LEN - 1);
75         *winner_score = best_exited_score;
76         *draw = exited_tied;
77     } else {
78         strncpy(winner_nick, best_any_nick, MAX_NICK_LEN
79               - 1);
80         *winner_score = best_any_score;
81         *draw = any_tied;
82     }
83     return GAME_OVER_TIMEOUT;
84 }
85
86 /* la partita continua */
87 return GAME_RUNNING;
88 }

```

Listing 33: Funzione game_check_end in game.c

```
1 void game_build_end_msg(const char *winner_nick, int
   winner_score, int draw, char *buf) {
2     if (draw)
3         snprintf(buf, MAX_MSG_LEN, "GAME_END DRAW %d",
                   winner_score);
4     else
5         snprintf(buf, MAX_MSG_LEN, "GAME_END WIN %s %d",
                   winner_nick, winner_score);
6 }
```

6.7.3 Cleanup, Disconnessione e Rigenerazione del Labirinto

Gestione della Disconnessione

La gestione della disconnessione di un client costituisce una fase fondamentale del ciclo di vita della partita, poiché la terminazione di una connessione non comporta esclusivamente la chiusura del relativo socket, ma richiede anche l'aggiornamento coerente dello stato condiviso del server.

Cleanup delle Risorse

In particolare, quando un giocatore abbandona la partita, il server deve provvedere alla corretta rimozione delle risorse associate al thread e alla connessione, alla disattivazione dello slot corrispondente nella `PlayerTable` e all'aggiornamento dei contatori relativi ai giocatori presenti e pronti. Tali operazioni devono essere eseguite in modo sincronizzato, in quanto lo stato del gioco è condiviso tra i diversi thread del server.

Nota

A differenza di una funzione che chiuda direttamente il socket, la `client_cleanup()` reale **non** riceve né il file descriptor né il nickname, e **non** chiude la connessione: si limita a rilasciare lo slot in `PlayerTable` e a segnalare l'evento sul `notify_pipe`. Questo perché, come descritto in §3.2, il socket deve poter restare aperto per un'eventuale rivincita (**AGAIN**): la chiusura effettiva avviene solo in `handle_client()`, quando il client non invia **AGAIN** entro il ciclo di rivincita.

Rigenerazione Automatica del Labirinto

Quando l'ultimo giocatore attivo si disconnette (`pt.count == 0`), il server considera conclusa la sessione corrente e **rigenera automaticamente** un nuovo labirinto tramite `maze_generate()`, azzerando anche `game_started` e il contatore di barriera `all_done`. Il server resta quindi sempre pronto ad accogliere una nuova partita senza necessità di riavvio del processo.


```
1 void client_cleanup(int player_idx, int* player_ready) {
2     if (player_idx >= 0) {
3         pthread_mutex_lock(&mutex);
4         if (*player_ready) pt.ready_count--;
5         *player_ready = 0;
6         client_fds[player_idx] = -1;
7         player_remove(&pt, player_idx);
8         broadcast_lobby_update();
9         if (pt.count == 0) {
10             pt.ready_count = 0;
11             maze.game_started = 0;
12             all_done = 0;
13             maze_generate(&maze);
14             log_write("maze regenerated (%dx%d)", MAZE_ROWS,
15                     MAZE_COLS);
16         }
17         pthread_mutex_unlock(&mutex);
18
19         write(notify_pipe[1], "\x01", 1);
20     }
21 }
```

Listing 34: Funzione client_cleanup completa (server.c)

Nota

Se il giocatore disconnesso era in stato `ready`, il contatore `pt.ready_count` viene decrementato: questo evita che una partita resti bloccata in attesa di un giocatore che non è mai stato effettivamente pronto (o che si è disconnesso dopo esserlo stato), e mantiene coerente il rapporto `ready_count/count` mostrato agli altri client in lobby tramite `WAITING`.

6.8 Logging Thread-Safe

Nell'esecuzione, più thread possono invocare `log_write()` in contemporanea. Per garantire l'atomicità di ogni riga di log, `logger.c` utilizza un proprio mutex interno indipendente dagli altri:

```
1 FILE *log_fp = NULL;
2 pthread_mutex_t log_mutex = PTHREAD_MUTEX_INITIALIZER;
3
4 void log_write(const char *fmt, ...) {
5     if (!log_fp) return;
6
7     time_t now = time(NULL);
8     struct tm *t = localtime(&now);
9     char ts[32];
10    strftime(ts, sizeof(ts), "%Y-%m-%d %H:%M:%S", t);
11
12    pthread_mutex_lock(&log_mutex);
13    fprintf(log_fp, "[%s] ", ts);
14    va_list ap;
15    va_start(ap, fmt);
16    vfprintf(log_fp, fmt, ap);
17    va_end(ap);
18    fputc('\n', log_fp);
19    fflush(log_fp);
20    pthread_mutex_unlock(&log_mutex);
21 }
```

Listing 35: Logging thread-safe in `logger.c`

7 Rendering del Client

7.1 Colori ANSI

Cella	Testo	Sequenza ANSI
Muro	—	\xf0\x91\x80\xa9
Libera	—	—
Oggetto	Giallo	\033[33m\033[1m @
Uscita	Verde	\033[42m
Inesplorata	—	?
Giocatore	—	\xe1\x8c\xb8

7.2 Terminale in Modalità Raw e Schermo Alternativo

Oltre alla resa cromatica, il client predispone il terminale per un'esperienza di gioco reattiva stile TUI (*text user interface*):

- **Modalità raw** (`enable_raw_mode()`, basata su `termios`): disattiva i flag `ICANON` ed `ECHO` sullo standard input, così che ogni tasto premuto (w/a/s/d/l/q/r) venga letto e inviato al server *immediatamente*, senza attendere l'Invio e senza eco automatico a schermo.
- **Schermo alternativo** (`ANSI_ALT_SCREEN_ON/OFF`): il client passa al buffer di schermo alternato del terminale, ripristinando quello originale alla chiusura, così da non "sporcare" la cronologia della shell con i redraw continui della mappa.
- **Thread timer dedicato**: un thread POSIX separato decrementa ogni secondo un contatore `time_remaining`, usato per mostrare il tempo residuo di partita senza bloccare il loop principale di I/O.
- Le sequenze di escape (freccie direzionali, ecc.) vengono lette e scartate esplicitamente, per evitare che vengano interpretate come comandi di gioco.

7.3 Gestione Asincrona dei Messaggi

```

1 while (running) {
2     fd_set fds;
3     FD_ZERO(&fds);
4     FD_SET(STDIN_FILENO, &fds);
5     FD_SET(sock, &fds);
6     select(sock + 1, &fds, NULL, NULL, NULL);
7     if (FD_ISSET(sock, &fds)) {
8         read_line(sock, line, sizeof(line));
9         running = handle_server_msg(line);
10    }
11    if (running && FD_ISSET(STDIN_FILENO, &fds)) {
12        char ch;
13        read(STDIN_FILENO, &ch, 1);
14        /* mapping tasto -> comando, diverso in lobby e in
15           partita */
16    }
17 }

```

Listing 36: Loop di gioco del client con select() (client.c)

Nota

A fine partita, dopo aver ricevuto **GAME_END**, il client invia automaticamente **AGAIN** al server per richiedere una nuova partita sulla stessa connessione, senza alcuna interazione richiesta al giocatore.

8 Parametri di Configurazione

Costante	Valore Default	Significato
MAZE_ROWS	20	Righe della mappa
MAZE_COLS	20	Colonne della mappa
VIEW_RADIUS	1	Raggio vista locale ($\Rightarrow$ finestra 3×3)
GLOBAL_MAP_INTERVAL	5	Secondi tra invii della mappa globale
GAME_TIMEOUT	90	Durata massima partita (secondi)
MAX_PLAYERS	10	Numero massimo di giocatori simultanei
MAX_NICK_LEN	32	Lunghezza massima del nickname
MAX_PASS_LEN	32	Lunghezza massima della password
MAX_MSG_LEN	1024	Lunghezza massima di un messaggio
NUM_OBJECTS	10	Oggetti raccogliibili nella mappa
NUM_EXITS	2	Uscite dal labirinto

9 Repository GitHub

Il progetto è disponibile pubblicamente su GitHub al seguente link:

github.com/TrialShock26/Labirinto

La repository contiene:

- L'intero codice sorgente sviluppato
- Il presente documento di progetto

Fine del Documento

Labyrinth Game

Documentazione Sistema Client–Server

Laboratorio di Sistemi Operativi

Si ringrazia il lettore per l'attenzione

Mario Majorano	N86005035
Luca Sanselmo	N86005147

Anno Accademico 2025/2026



UNIVERSITÀ DEGLI STUDI
DI NAPOLI FEDERICO II